

Introduction au Logiciel R

Jawad Boutarfass

Département des mathématiques
Faculté des Sciences et Techniques de Fès
Université Sidi Mohamed Ben Abdellah

Année universitaire 2025–2026

Objectifs du TP

La présente formation est destinée aux étudiants du tronc commun de la licence Sciences et Techniques à la FST de Fès, dans le cadre du module *Enquêtes et Techniques de Sondage / Data Processing*. Elle a pour objectif de fournir une introduction solide au logiciel R et de constituer un point de départ pour une utilisation autonome.

À l'issue de cette formation, l'étudiant sera capable :

- de saisir, organiser et manipuler des données dans R ;
- d'effectuer des analyses statistiques descriptives ;
- de réaliser différents types de graphiques ;
- d'utiliser les fonctions essentielles du logiciel pour l'estimation statistique.

Plan

Dans cette formation, nous allons suivre le plan suivant pour progresser pas à pas dans l'apprentissage du logiciel R :

- **TP1** : Introduction à R.
- **TP2** : Analyse statistique descriptive et réalisation de graphiques.
- **TP3** : Estimation statistique.

Section 1

Introduction à R

Qu'est-ce que R ?

- R est un système d'analyse statistique et graphique développé dans les années 1990 par Ross Ihaka et Robert Gentleman. Il constitue à la fois un logiciel et un langage de programmation.
- Logiciel libre et gratuit.
- Son utilisation est à la fois simple, flexible et très puissante.

Installation de R

- Télécharger R : <https://cran.r-project.org>
- L'installation de R est très simple, il suffit de suivre les instructions.
- Il existe des versions : Windows, MacOS X, Linux...

L'environnement de R

L'interface de R (RGui $\implies$ R **G**raphical **U**ser **I**nterface) sous Windows contient principalement les fenêtres suivantes :

- **Console R :**
 - Fenêtre principale avec le prompt `>`.
 - Permet d'exécuter directement des commandes.
- **Éditeur de scripts :**
 - Permet d'écrire, modifier et sauvegarder du code R.
 - Les fichiers sont enregistrés avec l'extension `.R`.
- **Fenêtre Graphics :**
 - S'ouvre automatiquement lors de la création d'un graphique.
 - Exemple : `plot(1:10)`.
- **Fenêtre Help :**
 - Affiche l'aide et la documentation des fonctions.
 - S'ouvre via fonction ou `help.start()`.

L'environnement de R

The screenshot shows the R GUI interface with several annotations in green text and black arrows:

- menus**: Points to the menu bar at the top (Fichier, Édition, Voir, Div, Paquets, Fenêtres, Aide).
- boutons**: Points to the toolbar below the menu bar.
- Fenêtre de graphiques**: Points to the plot window on the right, which displays a normal distribution curve titled "Densité normale $N(0,1)$ ".
- Invite de commandes**: Points to the R Console window on the left, which shows the R version (4.5.1), copyright information, and a prompt (`> |`).
- fenêtre de commandes**: Points to the same R Console window.

The R Console window contains the following text:

```
R version 4.5.1 (2025-06-13 ucrt) -- "Great Square Root"
Copyright (C) 2025 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R est un logiciel libre livré sans AUCUNE GARANTIE.
Vous pouvez le redistribuer sous certaines conditions.
Tapez 'license()' ou 'licence()' pour plus de détails.

R est un projet collaboratif avec de nombreux contributeurs.
Tapez 'contributors()' pour plus d'information et
'citation()' pour la façon de le citer dans les publications.

Tapez 'demo()' pour des démonstrations, 'help()' pour l'aide
en ligne ou 'help.start()' pour obtenir l'aide au format HTML.
Tapez 'q()' pour quitter R.

[Sauvegarde de la session précédente restaurée]

> |
```


Le fonctionnement de R

Une fois R installé, lancez l'exécutable correspondant pour démarrer le programme. L'invite de commandes (par défaut le symbole `>`) apparaît, indiquant que R est prêt à exécuter vos commandes.

- `>` : prompt principal, en attente de commande.
- `+` : indique qu'une commande n'est pas terminée.
- `#` : tout le reste de la ligne est un commentaire et ne sera pas interprété par R.

Opérations élémentaires dans R

```
> # Opérations élémentaires dans R
> #Addition
> 2+5
[1] 7
> #Soustraction
> 2-5
[1] -3
> #Division
> 10/2
[1] 5
> #Puissance
> 2^3
[1] 8
> #Racine carrée
> sqrt(25)
[1] 5
> |
```

Remarque : Chaque opération peut être utilisée directement dans la console R, et le résultat est affiché après avoir appuyé sur Entrée.

Les objets dans R

Dans R, un **objet** est une donnée stockée en mémoire. Les objets permettent de garder en mémoire des valeurs pour les réutiliser plus tard. On crée un objet en utilisant l'un des opérateur d'affectation "<-", "->" ou "=".

```
> x <- 3.143 # créé objet x en lui donnant la valeur
> y = x # créé objet y qui reçoit la valeur de x
> x = y # x reçoit la valeur de y
> x -> y # y reçoit la valeur de x
```

Le nom d'un objet doit obligatoirement commencer par une lettre (A–Z et a–z). R distingue les majuscules et les minuscules : x et X peuvent servir à nommer des objets distincts.

Les modes des objets dans R

Dans R, chaque objet possède un **mode**, c'est-à-dire le type de données qu'il contient. Voici les principaux modes :

- **numeric** : nombres réels ou entiers (1 , 2.33 , π) ;
- **complex** : nombres complexes (i , $1 + 2i$) ;
- **character** : chaînes de caractères ("K", "Hello") ;
- **logical** : valeurs booléennes (TRUE, FALSE)
- **NULL** : objet vide (NULL).

Modes des objets dans R

Pour connaître le mode d'un objet x de R, il suffit d'exécuter la commande `mode(x)` :

```
> mode(3.14)
[1] "numeric"
x<- 2+5i
> mode(x)
[1] "complex"
> mode("Bonjour")
[1] "character"
> mode(TRUE)
[1] "logical"
> mode(NULL)
[1] "NULL"
```

Les objets dans R

Pour afficher la valeur d'un objet `x`, on peut utiliser la commande `print(x)` :

```
> x = 2.71  
> print(x)      # pour afficher la valeur de x  
[1] 2.71
```

Ou, plus simplement, taper le nom de l'objet :

```
> x <- "Bonjour" # objet de type caractère  
> x  
[1] "Bonjour"
```

Lister les objets dans l'environnement R

Pour **lister tous les objets** présents dans l'environnement de travail (workspace), on peut utiliser la fonction `ls()` ou `objects()`. C'est très utile pour savoir quelles variables ou fonctions sont actuellement stockées en mémoire.

Exemple :

```
> x <- 3
> y <- "Bonjour"
> z <- 2 + 1i
> ls()
[1] "x" "y" "z"
```

Supprimer des objets dans R

La fonction `rm()` permet de supprimer des objets de l'environnement de travail.

Pour supprimer un objet `x`, on tape :

```
> rm(x)
```

Et pour en supprimer plusieurs objets :

```
> rm(objet1, objet2, objet3)
```

Pour supprimer tous les objets :

```
> rm(list = ls())
```


Supprimer des objets dans R

Et enfin, pour supprimer une liste d'objets qui possèdent une partie de leur nom en commun, par exemple une lettre spécifique, on utilise :

```
> lambda <- 1  
> alpha  <- 2  
> gamma  <- 3  
> rm(list = ls(pattern = "m")) # supprime lambda et  
  gamma
```

Les objets dans R

Avec R, les résultats d'une analyse peuvent même être stockés dans un objet, de sorte qu'une analyse peut être effectuée sans qu'aucun résultat ne soit affiché. Grâce à cet avantage, l'utilisateur peut ensuite extraire uniquement la partie des résultats qui l'intéresse :

```
> a=pi+sqrt(2) # stocke le résultat d'une opération
dans a
> b=exp(a)+10^-1
> c=a+b # stocke la somme de a et b
> c
[1] 99.83927
```

Les vecteurs dans R

Un **vecteur** dans R est un objet qui contient une **série d'éléments du même type** (numérique, caractère ou logique). C'est la structure de base la plus utilisée pour stocker des données en R.

La construction d'un vecteur

Il existe plusieurs façons de construire un vecteur. **La fonction collecteur** `c()` permet de regrouper plusieurs éléments dans un vecteur, en les séparant par des virgules.

```
> Age <- c(6, 40, 65, 35, 71, 10) # vecteur numérique
> Age
[1] 6 40 65 35 71 10
> Statut <- c("bébé", "enfant", "adulte", "vieux") #
  vecteur caractère
> Statut
[1] "bébé" "enfant" "adulte" "vieux"
> Actif <- c(TRUE, FALSE, TRUE, TRUE, FALSE, TRUE) #
  vecteur logique
> Actif
[1] TRUE FALSE TRUE TRUE FALSE TRUE
```

La construction d'un vecteur

L'opérateur «:» permet de créer une séquence de nombres entiers consécutifs

```
> v <- 1:5  
> v  
[1] 1 2 3 4 5
```

Construction par la fonction `seq()`

```
> u-> seq(1, 5)  
>u  
[1] 1 2 3 4 5  
> v -> seq(0, 1, by=0.2)  
>v  
[1] 0.0 0.2 0.4 0.6 0.8 1.0
```

La construction d'un vecteur

La fonction `rep()` répète des valeurs ou vecteurs.

```
> x <- rep('A', 5)
> x
[1] "A" "A" "A" "A" "A"
> y <- rep(c(1,2), 3)
> y
[1] 1 2 1 2 1 2
> z <- rep(c(1,2), each=3)
> z
[1] 1 1 1 2 2 2
```

Construire des vecteurs avec paste()

La fonction `paste()` permet de **générer des vecteurs de chaînes de caractères** en combinant des mots, des lettres ou des valeurs numériques :

```
> paste(c("A", "B", "C"), c(1, 2, 3))
[1] "A 1" "B 2" "C 3"
> paste("Etudiant", 1:5)
[1] "Etudiant 1" "Etudiant 2" "Etudiant 3" "Etudiant 4"
    "Etudiant 5"
> paste("Etudiant", 1:5, sep="")
[1] "Etudiant1" "Etudiant2" "Etudiant3" "Etudiant4" "
    Etudiant5"
> paste("Groupe", 1:3, sep = "_")
[1] "Groupe_1" "Groupe_2" "Groupe_3"
```

Quelques actions sur les vecteurs

Pour calculer la longueur d'un vecteur on utilise la fonction `length()` :

```
> a <- rep(c(1,2), each=3)
> length(a)
[1] 6
```


Tri des éléments d'un vecteur en R

La fonction `sort()` permet de trier les éléments d'un vecteur.

- `decreasing = FALSE` : tri croissant (par défaut)
- `decreasing = TRUE` : tri décroissant

```
> x <- c(12, 5, 8, 3, 14)
> sort(x)
[1] 3 5 8 12 14
> sort(x, decreasing = TRUE)
[1] 14 12 8 5 3
```

Tri des éléments d'un vecteur en R

- La fonction `order()` renvoie les *indices* du vecteur trié.
- Pour trier un vecteur `u`, on tape `u[order(u)]`.

```
> u <- c(12, 5, 8, 3, 14)
> order(u)
[1] 4 2 3 1 5
> u[order(u)]
[1] 3 5 8 12 14
```

Sélection d'une partie d'un vecteur

L'accès à une partie des éléments d'un vecteur s'opère avec l'opérateur de sélection `[ ]` et un vecteur de sélection :

```
> v <- 1:15  
> v[1]           # donne premier élément  
> v[2:4]         # éléments 2 à 4  
> v[c(1,3)]      # éléments 1 et 3  
> v[c(6,6,1:2)]  # donne les 6ème, 6ème, 1er et 2ème éléments de v  
v[10:1] # donne les 10ème, 9ème, ..., 1er éléments de v
```

On peut aussi sélectionner par des entiers négatifs :

```
> v[-(1:6)] # donne v sans ses 6 premiers éléments  
> v[-c(1,5)] # donne v sans le premier et le cinquième  
élément
```

La sélection par une condition logique :

```
> u=1:20  
> u[u>12]  
[1] 13 14 15 16 17 18 19 20  
u[(u<8)&(u==5)] & # signifie "et"  
[1] 5  
> u[(u<5)&(u>=12)]  
integer(0)  
> u[(u<5)|(u>=15)] # / signifie "ou"  
[1] 1 2 3 4 15 16 17 18 19 20
```

La fonction **which** renvoie l'indice d'un élément dans un vecteur :

```
> w=20:1  
> which(w==15)  
[1] 6
```

La fonction **all** renvoie TRUE si tous les éléments d'un vecteur satisfont une condition et FALSE sinon :

```
> x=c(1,1,2,1)  
> all(x==1)  
[1] FALSE
```

La fonction **any** renvoie TRUE dès que l'un des éléments satisfait la condition, FALSE sinon :

```
> any(x==2)  
[1] TRUE
```

Matrices dans R

Une **matrice** est un tableau rectangulaire composé de lignes et de colonnes, contenant des éléments de même type (numérique, logique, caractère...).

Dans R, une matrice est un **vecteur** auquel une structure bidimensionnelle (`dim`) est attribuée.

Créer une matrice dans R

La manière la plus courante de créer une matrice est d'utiliser la fonction `matrix()`. Elle permet de construire une matrice en précisant les éléments et le nombre de lignes ou de colonnes :

```
> m <- matrix(c(1,-4,2,10,6,0), nrow = 3)
> m
      [,1] [,2]
[1,]    1   10
[2,]   -4    6
[3,]    2    0
```

Créer une matrice dans R

Notons que, par défaut, R range les éléments dans une matrices par colonne. Pour ranger les éléments par ligne, il suffit d'utiliser l'argument `byrow = TRUE`.

```
> M <- matrix(1:6, ncol = 3, byrow = TRUE)
> M
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6

Construire une matrice avec `cbind()` et `rbind()`

La fonction `cbind()` permet de combiner des vecteurs en les mettant en colonnes :

```
> C1 <- c(1,2,3)
> C2 <- c(4,5,6)
> M <- cbind(C1, C2)
> M
```

	C1	C2
[1,]	1	4
[2,]	2	5
[3,]	3	6

Construire une matrice avec `cbind()` et `rbind()`

La fonction `rbind()` permet de combiner des vecteurs en les mettant en lignes :

```
> L1 <- c(1,0,-1)
> L2 <- c(4,-2,6)
> N <- rbind(L1,L2)
> N
```

	[,1]	[,2]	[,3]
L1	1	0	-1
L2	4	-2	6

Nommer les lignes et colonnes d'une matrice

Pour attribuer des noms aux lignes et aux colonnes d'une matrice, on utilise : `rownames()` pour les lignes et `colnames()` pour les colonnes :

```
> m <- matrix(1:9, nrow = 3, byrow = TRUE)
> rownames(m) <- c("Ligne1", "Ligne2", "Ligne3")
> colnames(m) <- c("Col1", "Col2", "Col3")
> m
```

	Col1	Col2	Col3
Ligne1	1	2	3
Ligne2	4	5	6
Ligne3	7	8	9

Nommer les lignes et colonnes d'une matrice

On peut aussi nommer lignes et colonnes en une seule commande :

```
> dimnames(m) <- list(c("L1", "L2", "L3"), c("C1", "C2", "C3"))
```

```
> m
```

	C1	C2	C3
L1	1	2	3
L2	4	5	6
L3	7	8	9

Sélection d'éléments dans une matrice

Dans une matrice, chaque élément est identifié par son numéro de ligne et de colonne. Pour accéder à l'élément (i, j) de la matrice m , on utilise :

```
> m[i,j]
```

Pour sélectionner la ligne i de la matrice m :

```
> m[i,]
```

Cela retourne la ligne i sous forme d'un vecteur. Pour extraire la ligne i sous forme d'une matrice uniligne :

```
> m[i,,drop=FALSE]
```

Sélection de colonnes et lignes dans une matrice

La commande :

```
> m[, c(3,3,1)]
```

retourne les colonnes 3, 3 et 1 de la matrice `m`; c'est donc une matrice à trois colonnes.

```
> m[-2,]
```

retourne la matrice `m` sans sa deuxième ligne.

```
> m[1:4, -2]
```

retourne les quatre premières lignes de `m` en excluant sa deuxième colonne.

```
> m[1:2, 2:3]
```

extrait de la matrice `m` la sous-matrice composée des lignes 1 et 2 et des colonnes 2 et 3.

Opérations sur les matrices

Fonction/Opérateur	Description
$A + B$, $A - B$	Addition/Soustraction terme à terme
$A * B$	Produit terme à terme
$A \%*\% B$	Produit matriciel
$t(A)$	Transposée de A
$diag(4)$	Matrice identité d'ordre 4
$det(A)$	Déterminant de A
$solve(A)$	Inverse de A
$solve(A,b)$	Résolution de système linéaire $Ax=b$
$eigen(A)$	Calcul des valeurs/vecteurs propres de A
$chol(A)$	Décomposition de Choleski
$qr(Y)$	Décomposition QR

Les data.frames

Un `data.frame`, ou jeux de données, est une structure de données en R permettant de stocker des données tabulaires.

- Chaque colonne peut contenir un type de données différent (`numeric`, `character`, `logical`, etc.).
- Toutes les colonnes doivent avoir la **même longueur**.
- Chaque ligne représente généralement un individu ou une observation.

Créer un data.frame

Pour créer un `data.frame`, on utilise la fonction `data.frame()`, qui permet de combiner des vecteurs de même longueur et éventuellement de types différents.

```
> Nom <- c("Ali", "Sara", "Lina", "Youssef")
> Age <- c(25, 30, 35, 40)
> Sexe <- c("M", "F", "F", "M")
> df <- data.frame(Nom, Age, Sexe)
> df
```

	Nom	Age	Sexe
1	Ali	25	M
2	Sara	30	F
3	Lina	35	F
4	Anas	40	M

Les data.frames

- Pour extraire des composantes d'un `data.frame`, on peut utiliser les méthodes présentées pour les matrices.
- Pour transformer une matrice en `data.frame`, on utilise la fonction `as.data.frame()`.
- Pour transformer un `data.frame` en matrice, on utilise la fonction `data.matrix()`.

Section 2

Statistique descriptive et réalisation de graphiques

Statistiques descriptive

R propose de nombreuses fonctions utiles en statistique. Les principales sont résumées dans le tableau ci-dessous :

<code>sum(x)</code>	somme des éléments de <code>x</code>
<code>prod(x)</code>	produit des éléments de <code>x</code>
<code>max(x)</code>	maximum des éléments de <code>x</code>
<code>min(x)</code>	minimum des éléments de <code>x</code>
<code>range(x)</code>	retourne le vecteur <code>c(min(x), max(x))</code>
<code>which.max(x)</code>	retourne l'indice du maximum des éléments de <code>x</code>
<code>which.min(x)</code>	retourne l'indice du minimum des éléments de <code>x</code>
<code>mean(x)</code>	moyenne des éléments de <code>x</code>
<code>median(x)</code>	médiane des éléments de <code>x</code>
<code>var(x)</code>	variance des éléments de <code>x</code>
<code>sd(x)</code>	l'écart-type de <code>x</code>

Statistiques descriptive

<code>cov(x, y)</code>	covariance entre deux variables x et y ou entre les colonnes de x et de y si ce sont des matrices ou des tableaux de données
<code>cov(x)</code>	covariance d'une matrice
<code>cor(x)</code>	matrice de corrélation si x est une matrice ou un tableau de données (1 si x est un vecteur)
<code>table(x)</code>	retourne un tableau des effectifs des différentes valeurs
<code>cumsum(x)</code>	un vecteur dont le $i^{\text{ème}}$ élément est la somme de $x[1]$ à $x[i]$
<code>cumprod(x)</code>	un vecteur dont le $i^{\text{ème}}$ élément est le produit de $x[1]$ à $x[i]$

Statistiques descriptive

<code>pmax(x,y,...)</code>	un vecteur dont le $i^{\text{ème}}$ élément est le maximum entre $x[i]$, $y[i]$, ...
<code>pmin(x,y,...)</code>	un vecteur dont le $i^{\text{ème}}$ élément est le minimum entre $x[i]$, $y[i]$, ...
<code>quantile(x)</code>	les quartiles de x
<code>quantile(x, 0.25)</code>	premier quartile de x
<code>quantile(x, 0.5)</code>	deuxième quartile (médiane) de x
<code>quantile(x, 0.75)</code>	troisième quartile de x

La fonction `summary()`

La fonction `summary()` fournit un résumé statistique automatique d'un objet R (vecteur, data frame...). Elle adapte les informations affichées selon le type de données.

Pour un vecteur numérique :

- Minimum
- 1er quartile (Q1)
- Médiane (Q2)
- Moyenne
- 3e quartile (Q3)
- Maximum

La fonction `summary()`

Pour un data frame :

- Résumé statistique pour chaque colonne numérique
- Comptage et affichage des modalités pour les variables qualitatives

Représentations graphiques dans R

R offre une large variété de fonctions graphiques pour représenter les données.

Les fonctions de base incluent :

- `plot()` : graphique générique (points, lignes...)
- `hist()` : histogramme
- `boxplot()` : boîte à moustaches
- `barplot()` : diagramme en barres
- `pie()` : diagramme circulaire

La fonction plot()

La fonction `plot(x,y)` sert à visualiser la relation entre deux variables quantitatives `x` et `y` :

```
> x=seq(-2*pi,2*pi,l=100)
> y=cos(x)
> plot(x,y) # un nuage de points : Figure 1
> plot(x,y, type="l") # courbe continue : Figure 2
> lines(x,sin(x),col="green")
```

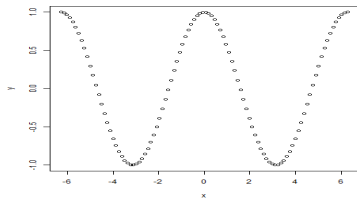


Figure – 1

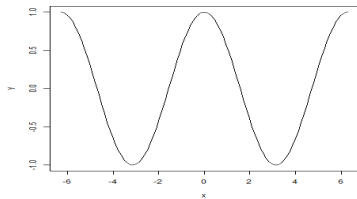


Figure – 2

La fonction plot()

```
> plot(x,cos(x), type="l", xlab="x", ylab="cos(x)",main="Graphe de fonction cosinus",col="blue")# Fig 3  
> abline(h=0, v=0, col="red") # ajouter les axes
```

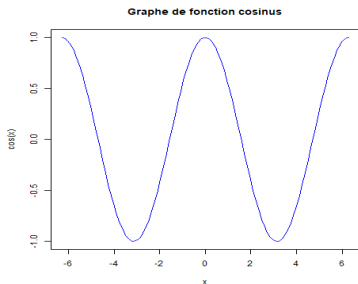


Figure – 3

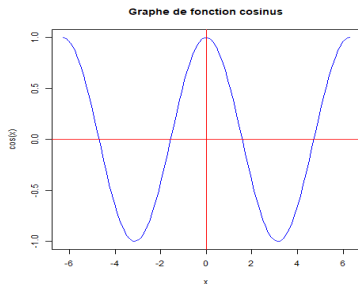


Figure – 4

La fonction plot()

```
> plot(x,cos(x), type="l", xlab="x", ylab="cos(x)",main="Graphe de fonction cosinus",col="blue")# Fig 3  
> abline(h=0, v=0, col="red")  
> lines(x,sin(x),col="green") Fig 5  
> points(0,0, pch="+", cex=3) Fig 6
```

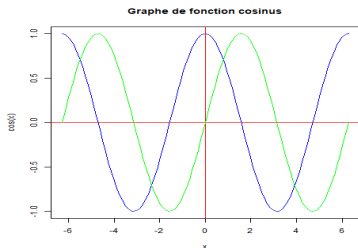


Figure – 5

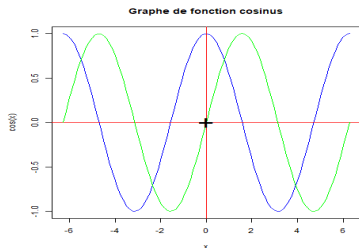


Figure – 6

Histogramme : hist()

La fonction `hist()` sert à représenter la distribution d'une variable quantitative :

```
> Notes=c(5,12,9,17,11,5.5,14.75,10,7,13,18)
> hist(Notes, main="Distribution des notes")
```

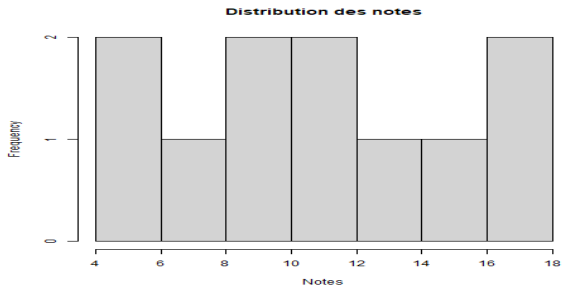


Figure – 7

Boîte à moustaches : boxplot()

La fonction `boxplot()` permet de visualiser la médiane, les quartiles et les valeurs extrêmes :

```
> Notes=c(5,12,9,17,11,5.5,14.75,10,7,13,18)
> boxplot(Notes, main="Boxplot des données")
```

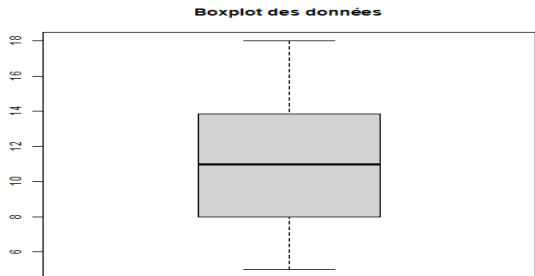


Figure – 8

Les fonctions barplot() et pie()

Les fonction barplot() et pie() utilisés pour représenter les fréquences d'une variable qualitative :

```
> Genre=c("H","F","F","H","F","H","H","F","F")  
> freq=table(Genre)  
> barplot(freq, col = c("red", "cyan"), main="Répartition par genre")  
> pie(freq, col = c("red", "cyan"), main="Répartition par genre")
```

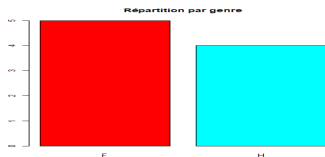


Figure – 11

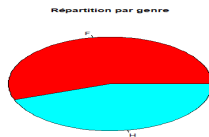


Figure – 11

Exporter un graphique dans R

Pour sauvegarder un graphique, on ouvre un fichier graphique (png, pdf, ...), on dessine le graphique, puis on ferme le fichier.

Exemple : export en PNG

```
> png("mon_graphique.png")  # crée un fichier image  
> plot(x, y, type = "l")     # tracer le graphique  
> dev.off()                  # ferme et enregistre le fichier
```

Le graphique est enregistré dans le répertoire de travail courant. Pour connaître ce dossier, on utilise :

```
getwd()  # affiche le dossier de travail courant
```